High-Efficient Fuzzy Querying With HiveQL for Big Data Warehousing

Bożena Małysiak-Mrozek, *Member, IEEE*, Jadwiga Wieszok, Witold Pedrycz, *Life Fellow, IEEE*, Weiping Ding, *Senior Member, IEEE*, and Dariusz Mrozek, *Senior Member, IEEE*

***Abstract*—Querying and reporting from large volumes of structured, semistructured, and unstructured data often requires some flexibility. This flexibility provided by fuzzy sets allows for categorization of the surrounding world in a flexible, human-mind-like manner. Apache Hive is a data warehousing framework working on top of the Hadoop platform for big data processing. Hive allows executing queries and aggregating and analyzing data stored in Hadoop distributed file system and other repositories. Hive responds to the current needs for efficient big data warehousing, which is impossible with traditional data warehouses due to their rigid nature. This article presents the FuzzyHive library that extends the Hive framework with fuzzy sets based techniques for querying, analyzing, and reporting on big data warehouses. We formalize the fuzzy techniques used while operating on Hive-based data warehouses (including fuzzy filtering on dimensional attributes, projection with fuzzy transformation, fuzzy grouping, and joining). We also show how we embedded these operations in Hive query language, which was not studied so far. Such extensions make big data warehousing more flexible and contribute to the portfolio of tools used by the community of people working with fuzzy sets and data analysis. The FuzzyHive library complements the spectrum of available solutions for fuzzy data processing and querying in large datasets. We investigate Hive fuzzy querying performance, effectiveness, and scalability for various data storage formats (text, Avro, and Parquet). Our experiments demonstrate that the proposed extensions introduce more elasticity and are also efficient for big data warehousing, which is the first such kind of solution for this environment.**

***Index Terms*—Big data, data warehousing, fuzzy sets, Hadoop, Hive, querying.**

Manuscript received December 3, 2020; revised March 11, 2021; accepted March 18, 2021. Date of publication March 29, 2021; date of current version June 2, 2022. This work was supported in part by Amazon Web Services grant under pro-quality Grant 02/100/RGJ21/0009, and the professorship under Grant 02/020/RGP19/0184 of the Rector of the Silesian University of Technology, Gliwice, Poland, in part by the Statutory Research funds of Department of Applied Informatics, Silesian University of Technology under Grant 02/100/BK_21/0008 and BK-221/RAU7/2021, in part by the National Natural Science Foundation of China under Grant 61300167 and Grant 61976120, in part by the Natural Science Foundation of Jiangsu Province under Grant BK20191445, and in part by the Qing Lan Project of Jiangsu Province, China. *(Corresponding author: Dariusz Mrozek.)*

Bożena Małysiak-Mrozek is with the Department of Graphics, Computer Vision, and Digital Systems, Silesian University of Technology, 44-100 Gliwice, Poland (e-mail: bozena.malysiak-mrozek@polsl.pl).

Jadwiga Wieszok and Dariusz Mrozek are with the Department of Applied Informatics, Silesian University of Technology, 44-100 Gliwice, Poland (e-mail: jadwiga.wieszok@gmail.com; dariusz.mrozek@polsl.pl).

Witold Pedrycz is with the Department of Electrical and Computer Engineering, University of Alberta, Edmonton, AB T6G 2V4, Canada, with the Department of Electrical and Computer Engineering, Faculty of Engineering, King Abdulaziz University, Jeddah 21589, Saudi Arabia, and also with the Systems Research Institute, Polish Academy of Sciences, 01-447 Warsaw, Poland (e-mail: wpedrycz@ualberta.ca).

Weiping Ding is with the School of Information Science and Technology, Nantong University, Nantong 226019, China (e-mail: dwp9988@163.com).

This article has supplementary material provided by the authors and color versions of one or more figures available at https://doi.org/10.1109/TFUZZ.2021.3069332.

Digital Object Identifier 10.1109/TFUZZ.2021.3069332

I. Introduction

In the last three decades, data warehousing has been an essential technique for collecting and managing data coming from different sources to provide efficient reporting and deliver meaningful business insights based on consolidated data. However, in the era of big data that we entered several years ago, the enormity of data (data *volume*), the speed data are generated (data *velocity*), and the heterogeneous, often unstructured nature of the data (data *variety*) caused many data analysis pipelines that do not fit traditional data warehousing methods. Although access to such a new class of datasets gives us an unprecedented opportunity to learn and understand the processes that govern the world around us, or even ourselves, it also raises many challenges and several technical problems. It has also changed many processes related to building systems for data warehousing. These changes involve moving from *fixed-schema* data warehouses to *schema-less* data lakes for storing data, from *extract, transform, and load* to *extract, load, and transform* procedures for data capturing and preparation, from *schema on write* to *schema on read* data analysis strategies, and from *scaling up* to *scaling out* solutions for increasing performance of analytic queries. Hadoop framework is one of the tools that, together with the Hadoop distributed file system (HDFS) and MapReduce processing model, enables us to collect, store, process, and analyze the data, solving or mitigating these technical problems.

Similar to the analytical querying performed against traditional database systems and data warehouses, querying in big data warehouses can benefit from including fuzzy terms in it. A typical application might involve finding similar cases while analyzing a specific problem with flexible filtering on dimensional attributes in a big data warehouse (e.g., cases of similar patients suffering from the same disease). Although there are ready solutions that allow fuzzy searching for items, e.g., in relational databases, most of them do not address the challenges and technical problems of big data. In particular, they do not work with the broad Hadoop ecosystem. Therefore, they do not support modern, big-data-oriented data warehousing by providing mechanisms that would give humanlike flexibility

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License. For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

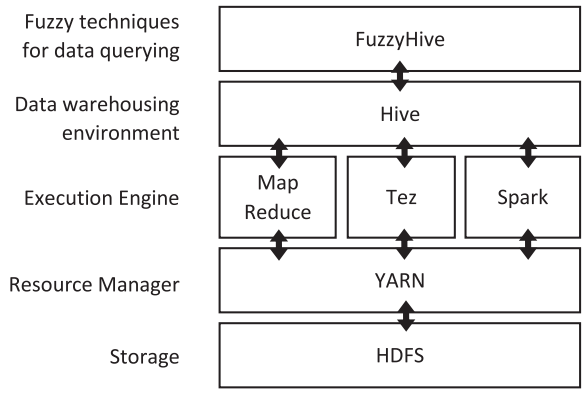

Fig. 1. Location of the FuzzyHive library in the Hadoop/Hive stack and dependencies between components.

in formulating elastic search criteria or abstract membership categories. This article addresses these problems by extending the Hive framework.

Hive is a framework for data warehousing built on top of Hadoop. It became one of the essential tools in the broad Hadoop ecosystem, which currently consists of a rich collection of tools and frameworks for different big data processing purposes. Hive queries and analyzes big data stored in the HDFS using structured query language (SQL) like query language, called HiveQL. HiveQL allows managing large volumes of unstructured data stored in the HDFS as structured and utilizing strong SQL skills of data analysts working so far with relational databases.

Big data warehousing is one of the timely areas in data analytics. Bringing concepts of fuzzy sets and their efficient realization with the aid of the HiveQL comes with a high level of originality. Evidently, this study could foster long-term and far-reaching studies augmenting data analytics by efficient implementation of constructs of fuzzy set technology. Within this article, we propose extensions to the modern, Big-data-oriented data warehousing performed with Hive (Fig. 1). These extensions, provided as FuzzyHive library residing on top of Hive, implement the benefits of using fuzzy logic in data processing, querying, and analysis. They enable flexible querying with HiveQL against large datasets in which searching criteria cannot be precisely defined, or their application would significantly narrow the result set. Some of the fuzzy techniques are already known from relational databases. Still, we formalize them for the dimensional attributes, apply them in the new environment for big data warehousing, embed them in the declarative language used for formulating analytical queries, and show how they scale in such an environment. Therefore, the main contributions of this article cover extensions to the HiveQL that support elastic filtering and grouping on dimensional attributes, fuzzy joining on attributes shared by many tables of a big data warehouse, and flexible ad hoc querying capabilities against the dynamically defined schema of a data warehouse built over a big data lake. As a result of our research, fuzzy methods successfully used in relational and NoSQL databases (but not adjusted for querying big data) are now applied for elastic data querying in a new big data warehousing environment. In this way, we show that these techniques can be well-integrated with a new environment for data analytics and reporting.

The rest of this article is organized as follows. Some related work is reviewed, and existing challenges are analyzed in Section II. In Section III, we provide formal definitions of the fuzzy operations provided within the fuzzy extension for Hive (FuzzyHive). Section IV shows implementation details of the FuzzyHive extension and main functions exposed and used in analytical HiveQL queries. Section V provides results of experiments related to flexible querying the growing volume of data in a Hive data warehouse. Finaly, Section VI concludes this article.

II. Related Works

Earlier works related to using fuzzy logic in data processing focus mainly on extending capabilities of relational database management systems and SQL toward fuzzy querying against relational tables. These works were realized in languages, such as SQLf [1], FQUERY [2], Soft-SQL [3], fuzzy generalized logical condition [4], FuzzyQ [5], and fuzzy SQL extensions for relational databases [6]–[8], enabling the use of fuzzy techniques, including filtering with fuzzy criteria, fuzzy grouping, defining fuzzy linguistic terms, and inference, for data retrieval, an essential milestone in fuzzy querying with SQL. However, these solutions were not focused on big data's challenges and cannot address them adequately.

Since traditional data warehousing relied upon relational databases, where all data necessary for making critical decisions were stored in databases with a dedicated schema (e.g., star or snowflake schema), and OLAP cubes to summarize data, some of the fuzzy querying techniques have also been implemented in this area. For example, Delgado *et al.* [9] and Molina *et al.* [10] proposed a fuzzy multidimensional model that can manage imprecision both in the dimensions and the facts. In one of our previous works [11], we defined a concept of the fuzzy data warehouse (in the traditional sense) as a data repository that may consist of fuzzy dimensions (dimensions with fuzzy attributes) and fuzzy facts (with fuzzy measures). Castillo-Ortega *et al.* [12] use fuzzy linguistic terms to organize the time dimension of OLAP cubes to improve the understandability of the results. Lara *et al.* [13] show the use of soft computing methods while building OLAP multidimensional models for learning object repositories. Fundamental concepts related to the construction, application, implementation, and evaluation of fuzzy data warehouses were presented by Fasel [14]. Finally, Martinez-Rojas *et al.* [15] and Ruiz *et al.* [16] show the use of fuzzy multidimensional modeling and fuzzy OLAP in cost analysis in construction projects and management of imprecision in construction tasks.

The rigid nature of data schema used in relational databases and traditional data warehouses, together with the exponential curve of costs for scaling the hardware, naturally shifted focus to NoSQL databases that provide more flexibility for schema changes and horizontal scalability. For example, Hindri *et al.* [17] proposed a fuzzy NoSQL model to deal with large fuzzy datasets stored as graphs in the Neo4j database. In the same graph database model, Castelltort and colleagues [18]–[21]

implemented fuzzy filtering and linguistic filtering, extending standard querying performed with the use of the Cypher query language. Kacem and Touzi proposed flexible querying with linguistic labels with Mongo query language for the MongoDB document database [22]. Those solutions focus mainly on the *volume* characteristic of the big data taking into account extensive capabilities for scaling and partitioning data in NoSQL databases. However, the authors have not provided any performance tests results that would prove their solution's efficiency.

In recent years, a broad spectrum of works has been devoted to the application of fuzzy techniques in reducing the massive volumes of data through clustering. This can be achieved by reducing the dimensionality of big data, such as reported by Azar and Hassanien in [23]. As a consequence, the reduction leads to the acceleration of the training phase while building neural-fuzzy classification models. The data reduction has also been achieved by discretizing conditional attributes and fuzzification of class labels, as proposed by Li *et al.* [24]. Solutions that rely on the Hadoop ecosystem, including Apache Hadoop and Apache Spark processing frameworks, are also used here to accelerate classification tasks with various algorithms, including fuzzy K-nearest neighbor classifier [25], [26], fuzzy-rule-based associative classifier [27], fuzzy-rule-based classification system [28], and its extensions [29], [30], fuzzy decision trees [31], and fuzzy cognitive maps and linguistic fuzzy cognitive maps [32]–[35]. Many studies in the field of big data, including the work in [36]–[45], adopt dedicated versions of fuzzy C-means algorithm to perform fuzzy clustering or distributed fuzzy clustering [46] on huge datasets (data at rest) or data streams (data in motion). Linguistic summarization through MapReduce-based data rewriting is shown in [47]. Although the solution presented in our article allows for fuzzy grouping with linguistic terms, the solution design operates on whole datasets, supporting efficient querying along various dimensions of performed data analysis.

This is possible by using the concept of a *data lake*, where data are stored in its native format and get structuralized when they are needed (the *schema on read* approach) [48]–[50]. Such an approach in processing data in the big data Hadoop environment is directly visible in the works of Khorasani *et al.* [51] and our previous works [52], [53]. Khorasani *et al.* designed fuzzy relational operations as MapReduce procedures executed on the Hadoop cluster. The solution is efficient for large datasets but is procedural (i.e., not declarative), which requires encoding each operation in a procedural language. In contrast, our solutions for fuzzy querying over data lake presented in [52] and [53] rely on declarative U-SQL language but use a commercial Data Lake Analytics system. Limiting the application of the fuzzy querying to the Azure cloud platform that the solutions were designed for, categorically excluded the possibility of using them on the hardware kept on-premises. The extension for the Apache Hive presented in the article hybridizes both approaches. It implements fuzzy operations in a declarative HiveQL language and performs fuzzy relational operations in the open-source Hadoop framework.

As discussed, although a great effort has been dedicated to fuzzy querying in relational databases, clustering, and classification of big data to solve large-scale analytics with dynamically defined data warehouses in the face of uncertainties, the noted algorithms suffer from the following limitations and challenges.

1) In majority, the mentioned solutions are not declarative, which does not allow for the easy formulation of queries supporting comprehensive analyses.
2) These solutions do not address big data warehousing, where query execution is distributed over many computer cluster nodes.

This article shows how fuzzy data analysis in data warehouses built on top of big data sets can be performed using the Hive environment. This article extends existing solutions by providing the following.

1) A developed model that allows fuzzy querying in big data sets for efficient data warehousing.
2) Data structures that facilitate the definition of domain-oriented fuzzy sets, linguistic variables, and categories.
3) Results of performance tests for extensive fuzzy searches executed in the Hadoop environment and investigations of the impact of various parameters on the query execution efficiency.

TABLE I
PART OF *PIMA_DIABETES* HIVE TABLE

Date	Pregnancies	Glucose	Blood Pressure	Skin Thicknes	Insulin	BMI	Pegdigree Function	Age	Dbets Class
2020-03-21	6	148	72	35	0	18.06	0.627	50	1
2015-06-03	8	183	64	0	0	23.32	0.672	32	1
2017-12-12	1	89	66	23	94	28.11	0.167	21	0
2014-01-25	0	137	40	35	168	43.16	2.288	33	1
2015-11-15	10	190	0	0	0	35.33	0.134	29	0
2020-09-06	5	116	74	0	0	25.60	0.201	30	0
2018-06-18	3	78	50	32	88	19.10	0.248	26	1
2019-04-30	1	85	66	29	0	17.22	0.351	31	0
2017-10-09	2	115	70	45	543	27.82	0.158	53	1

III. FUZZY OPERATIONS IN HIVEQL

Fuzzy querying in HiveQL requires data to be loaded in Hive and stored in the HDFS. Hive relies on the concept of tables as organizational units for data (such as in relational databases). However, the data are stored in files that may have different formats, sometimes complex and not row-oriented. In more straightforward cases, these can be comma-separated or tab-delimited files. It is relatively easy to map the structure of a created table to a row-oriented file, as each row of the file is extracted and seen as a tuple of the table. The data warehouse structure is usually also flat compared to the traditional relational data warehouses designed according to star or snowflake schemas. This is not only because of the flat format of the data files the data are stored in, but also because of avoiding the costly join operations that would complicate the execution of analytical queries translated to underlying MapReduce jobs (or DAGs, i.e., direct acyclic graphs). Therefore, the multidimensional model of the data, including cubes, dimensions, and facts, is built based on the flat structure of the Hive table. Given is a relation R representing Hive table with m crisp attributes

$$R = \{R_1, R_2, R_3, \ldots, R_m\}. \tag{1}$$

TABLE II
PART OF *NHS_SURVEY* HIVE TABLE

Gnd	Age	HT	WT	Waist	Pulse	SYS	DIAS	CHOL	BMI	LEG	Elbow	Wrist	Arm	BMI_CAT
F	17	64.3	114.8	67.2	76	104	61	264	19.6	41.6	6	4.6	23.6	Normal
F	32	66.4	149.3	82.5	72	99	64	181	23.8	42.8	6.7	5.5	26.3	Normal
F	19	64.8	105.4	72.9	76	112	62	44	17.7	39.1	5.7	4.8	23.7	Underweight
F	55	62.3	160.1	93	60	114	76	384	29.1	40.2	6.2	5	32.6	Overweight
F	27	59.6	127.1	82.6	72	94	58	98	25.2	36.2	5.5	4.8	29.2	Overweight
F	53	68.7	214.5	107.5	56	125	84	288	32.1	42.8	8.2	5.9	37.6	Obese
F	57	63.4	255.9	126.5	80	155	85	600	43.9	41	8	5.6	43.8	Obese

For example, Table I shows sample tuples (records) of the relation (table) *pima_diabetes* that stores information on the occurrence of diabetes in women of Pima Indian heritage. The data were loaded into the Hive table from UCI Machine Learning Pima Indians Diabetes Database [54]. Table II shows sample tuples of the relation *nhs_survey*. Data in this Hive table were loaded from the Third National Health and Nutrition Examination Survey dataset [55] collected by the United States Department of Health and Human Services, NHS. Both relations will be used in several examples presented throughout this article.

The big data cube C is defined as follows:

$$C = \langle D, M \rangle \tag{2}$$

where D is a set of dimensions defined over the Hive table R, and M is the set of measures. The schema of cube C is a composition of schemata of dimensions and measures

$$\chi_C = \chi_D \cup \chi_M. \tag{3}$$

Each dimension D_i is defined as a set of attributes A and hierarchies of attributes H

$$D_i = \langle A, H \rangle. \tag{4}$$

There exists a functional dependence between attributes of dimensions and attributes of the Hive table $A \longrightarrow R$. Additionally, $H \subseteq A$ and usually $M \subset R$. For example, we can define a dimension *Patient* over the *pima_diabetes* Hive table with *DbetesClass*, *Age*, and *BMI* attributes and $DbetesClass \rightarrow BMI$ hierarchy of attributes. Based on the same table, we can define the *Time* dimension with *Year*, *Quarter*, *Month*, *Day* attributes, and $Year \rightarrow Month \rightarrow Day$ and $Quarter \rightarrow Month$ hierarchies (all extracted from the *Date* attribute on data reading). Measures can be defined on the *Glucose* and *Insulin* attributes of the Hive table (with the *Average* as the aggregation function). One of the measures can also be *the number of patients*, which is calculated by counting the rows in analyzed groups of tuples, but is not present in the underlying Hive table.

Fuzzy querying in such a big data warehouse environment requires extending the relational algebra to perform the following operations: fuzzy selection, projection with fuzzy transformation, fuzzy grouping, fuzzy join, building complex fuzzy filtering criteria, and defining linguistic variables and linguistic values. This section gives formal details of the aforementioned operations that we have designed and implemented in the HiveQL.

Given is the dimension D with n crisp attributes (we will consider only the attribute part of the dimension $D = \langle A \rangle$)

$$D = \{A_1, A_2, A_3, \ldots, A_n\}, D \subset R. \tag{5}$$

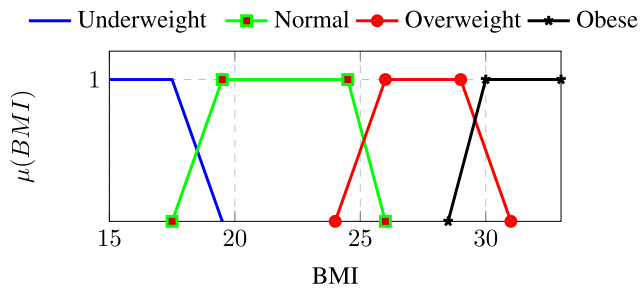

Fig. 2. Graphic representation of the linguistic values of the *vBMI* linguistic variable.

A. Fuzzy Selection on Dimensional Attributes

Fuzzy selection $\tilde{\sigma}$ produces a dimension D', which is a subset of the dimension D, on the basis of fuzzy filtering condition

$$D' = \tilde{\sigma}_{A_i \overset{\lambda}{\approx} \phi}(D) = \{t : t \in D, \mu_\phi(t(A_i)) \geq \lambda\} \tag{6}$$

where $A_i \overset{\lambda}{\approx} \phi$ is a fuzzy filtering condition, A_i is an attribute of the dimension D, ϕ is a fuzzy set (e.g., *BMI around 28* or *BMI is normal*) represented by a membership function μ_ϕ, $\approx$ is an operator that compares the numeric value of the attribute A_i for each tuple t from the dimension D with the fuzzy set ϕ, $t(A_i)$ stands for the value of the attribute A_i for the tuple t, λ is a minimum membership degree (threshold) for which the fuzzy filtering condition is satisfied, and μ_ϕ is a membership function of a fuzzy set ϕ.

For example, results of the fuzzy selection on the *Patient* dimension $\tilde{\sigma}_{\text{BMI} \overset{0.7}{\approx} \sim 28}(\text{Patient})$, which implements the following query formulated in natural language: *Find all patients with a BMI around 28*, are shown below. The last column of the result set provides the matching (compatibility or membership) degree between the value of the BMI attribute for the current tuple and the fuzzy set *BMI around 28*.

```
DbtesClass BMI   MemDg
---------- ----- -----
0          28.11 0.926
1          27.81 0.873
```

The SQL-like form of the query would look like as follows (*around 28* fuzzy set is defined by the trapezoidal/triangular membership function):

```
SELECT DbtesClass, BMI,
  around(BMI, 26.5, 28, 28, 29.5) AS MemDg
FROM Patient
WHERE (BMI around 28) >= 0.7
```

However, in HiveQL, this query would be represented by query Q1 shown in Table III and its usage requires knowledge of the implemented modules introduced in Section IV.

We also provided the possibility to filter dimensional data based on fuzzy filtering criteria formulated with linguistic values of predefined linguistic variable $L = \{l_1, l_2, \ldots, l_v\}, v \in \mathbb{N}_+$. Linguistic values are fuzzy sets defined by an expert (e.g., *underweight*, *normal*, *overweight*, *obese BMI*, as in Fig. 2). For

TABLE III
QUERIES USED IN PERFORMANCE TESTS

Id	Category	HiveQL Query
Q1	Fuzzy filtering	`select * from pima_diabetes where around(bmi,26.5,28,28,29.5) > 0.7;`
Q2	Filtering with a linguistic variable	`select * ,bmi_underweight(bmi) from pima_diabetes where bmi_underweight(bmi) > 0.7;`
Q3	Grouping with a linguistic variable	`select bmi_gr.bmi_group, count(*) as count from (select *, bmiToLing(bmi) as bmi_group from pima_diabetes) bmi_gr group by bmi_gr.bmi_group;`
Q4	Filtering with a simple logical expression	`select * from pima_diabetes where fuzzyor(around(bmi, 27,28,28,29), around(age, 45,50,50,55)) > 0.7;`
Q5	Filtering with complex logical expressions	`select * from pima_diabetes where fuzzyor(fuzzyand(bmi_normal( bmi), glucose_normal(glucose)), fuzzyor(bmi_obese(bmi), bmi_underweight(bmi)))> 0.7;`
Q6	Fuzzy join with a linguistic variable	`select * from (select *, bmiToLing(bmi) gr from pima_diabetes) data join nhs_survey r on data.gr = r.bmi_cat ;`
Q7	Fuzzy join through fuzzy numbers	`select diabetes.bmi, survey.bmi, fequals(diabetes.bmi, 2, survey.bmi, 1) as memberDegree, diabetes.glucose as glucose, diabetes.insulin as insulin, survey.chol as cholesterol, survey.pulse as pulse from pima_diabetes_1000 diabetes join (select * from nhs_data) survey where fequals(diabetes.bmi, 2, survey.bmi, 1)> 0.7;`

example, the results of the fuzzy selection on the *Patient* dimension $\tilde{\sigma}_{\text{BMI}\overset{0.7}{\approx}\text{Underweight}}(\text{Patient})$, which implements the following query formulated in natural language: *Find all patients who are underweight*, are shown below. The last column of the result set provides the matching (compatibility or membership) degree between the value of the BMI attribute for the current tuple and the fuzzy set *Underweight BMI*.

```
Outcome BMI   MemDg
------- ----- -----
1       18.06 0.735
0       17.21 1.000.
```

The SQL-like form of the query would look like as follows:

```
SELECT DbtesClass, BMI,
  bmi_undeweight(BMI) AS MemDg
FROM Patient
WHERE (BMI is Underweight) >= 0.7.
```

In HiveQL, this query, represented by query Q2 shown in Table III, contains an invocation of dedicated functions introduced in Section IV.

B. Combining Logical Conditions on Dimensional Attributes

Filtering criteria in fuzzy selection on dimensional attributes can be complex and may consist of several fuzzy conditions combined with the use of fuzzy operators of conjunction and disjunction. In our extension to Hive (called FuzzyHive), these operators are implemented according to t-norm (minimum) and t-conorm (maximum)

$$\tilde{\sigma}^{\lambda}_{A_i\approx\phi_i\,\tilde{\wedge}\,A_j\approx\phi_j}(D) = \{t : t \in D, \min(\mu_{\phi_i}(t(A_i)), \mu_{\phi_j}(t(A_j))) \geq \lambda\} \quad (7)$$

where $\tilde{\wedge}$ is the symbol used to denote logical conjunction in fuzzy logic, A_i and A_j are attributes of the dimension D, $i, j = 1\ldots n; i \neq j$, ϕ_i and ϕ_j are fuzzy sets; λ is a minimum membership degree for which the whole compound fuzzy condition is fulfilled, μ_{ϕ_i} and μ_{ϕ_j} are membership functions of fuzzy sets ϕ_i and ϕ_j, and

$$\tilde{\sigma}^{\lambda}_{A_i\approx\phi_i\,\tilde{\vee}\,A_j\approx\phi_j}(D) = \{t : t \in D, \max(\mu_{\phi_i}(t(A_i)), \mu_{\phi_j}(t(A_j))) \geq \lambda\} \quad (8)$$

where $\tilde{\vee}$ is the symbol used to denote logical disjunction in fuzzy logic.

For example, the table below shows results of the fuzzy selection on the patient dimension $\tilde{\sigma}^{0.7}_{\text{BMI}\approx\sim 28\,\tilde{\vee}\,\text{age}\approx\sim 50}(\text{Patient})$, which implements the following query formulated in natural language: *Find all patients with a BMI around 28 or age around 50*.

```
DbtesClass  BMI   age
----------  ----- ----
1           18.06   50
0           28.11   21
1           27.82   53.
```

The SQL-like form of the query would look like as follows:

```
SELECT DbtesClass, BMI, age
FROM Patient
WHERE (BMI around 28
  fuzzyOR age around 50) >= 0.7.
```

In HiveQL, this query would be represented by query Q4 shown in Table III. In FuzzyHive extension, the *fuzzyOR* operator is implemented as a function (introduced in Section IV), which accepts multiple arguments. Fuzzy conditions can be applied both on dimensional attributes and on measure attributes. Query Q5 in Table III shows an example of such a case.

C. Projection of Dimensional Attributes With Fuzzy Transformation

Projection with fuzzy transformation $\tilde{\Pi}$ produces a dimension D' with a restricted subset of attributes of the set of attributes of the dimension D, in which one (or more) of the attributes

Algorithm 1: Projection With Fuzzy Transformation.

for each value of the transformed attribute for the current tuple $t(A_k) : t \in D, k < n$ **do**
  $max \leftarrow 0$
  $label \leftarrow null$
  for each linguistic value $l_i, i = 1, \ldots, v$ **do**
    calculate membership degree $\mu_i(t(A_k))$
    if $\mu_i(t(A_k)) > max$ **then**
      $max \leftarrow \mu_i(t(A_k))$
      $label \leftarrow l_i$
    end if
  end for
  Replace value $t(A_k)$ by the $label$
  return $label$
end for

undergoes fuzzy transformation $\tilde{T_L}$ with the use of linguistic variable L

$$D' = \tilde{\Pi}_{A_1 A_2 \ldots \tilde{T_L}(A_i) \ldots A_k}(D) = \{t[A_1 A_2 \ldots \tilde{T}(A_i) \ldots A_k] : t \in D, k \leq n\} \quad (9)$$

where $\{A_1, A_2, \ldots, A_i, \ldots, A_k\} \subseteq \{A_1, A_2, A_3, \ldots, A_n\}, k \leq n$. The transformation $\tilde{T_L}$ returns a linguistic value (a label) l from a predefined linguistic variable $L = \{l_1, l_2, \ldots, l_v\}, v \in \mathbb{N}_+$. Fuzzy transformation is performed according to Algorithm 1. The algorithm finds the best matching between the current value of the transformed attribute A_i for the tuple t and the fuzzy sets representing linguistic values, and it returns the label of the fuzzy set that the crisp value fits the most. The time complexity of Algorithm 1 depends on the number of tuples N within the dimension (or dimensional table) and the number of linguistic values v that should be checked. Therefore, the theoretical time complexity is $O(Nv)$. Since, in most cases, v is low, the practical time complexity will be $O(N)$.

For example, the extended projection with fuzzy transformation $\tilde{\Pi}_{\text{BMI}, \tilde{T}_{\text{vBMI}}(\text{BMI}), \mu_{\text{vBMI}}(\text{BMI})}(pima_diabetes)$ on the basis of the *vBMI* linguistic variable, defined as presented in Fig. 2, produces a partial result set shown below. This operation assigns patients an appropriate label based on their BMI for a defined linguistic variable.

BMI	label	MemDg
18.06	Underweight	0.735
23.32	Normal	1.0
19.10	Normal	0.8
28.11	Overweight	1.0
43.16	Obese	1.0
35.33	Obese	1.0
25.60	Overweight	0.8
17.21	Underweight	1.0
27.81	Overweight	1.0

The SQL-like form of the query would look like as follows:

```
SELECT BMI, bmiToLing(BMI) AS label,
  membership(label, BMI) AS MemDg
FROM Patient
```

Algorithm 2: Fuzzy Grouping by Dimensional Attributes.

for each linguistic value $l_i, i = 1, \ldots, v$ **do**
  Create a group for l_i
end for
for each value of the grouping attribute for the current tuple $t(A_k) : t \in D, k < n$ **do**
  Transform the member $t(A_k)$ to a label
  Assign tuple t to appropriate group according to the achieved label
end for

Query Q3 in Table III shows an example of such a projection. The *bmiToLing* and *membership* functions are parts of the FuzzyHive extension explained in Section IV in more detail.

D. Fuzzy Grouping by Dimensional Attributes

Grouping is one of the basic operations applied in data warehousing since it underlies data aggregation performed while reporting on data stored in a data warehouse or big data lake. The grouping operation allows assigning members of a selected attribute A_k (the grouping attribute) of the dimension D to a particular group. However, unlike in the traditional grouping, in which every distinct member of the grouping attribute leads to the formation of a group, in fuzzy grouping, the groups are predefined by a domain expert and represented by fuzzy sets. The predefined fuzzy sets are labeled with linguistic values of a predefined linguistic variable $L = \{l_1, l_2, \ldots, l_v\}, v \in \mathbb{N}_+$. The grouping operation is performed according to Algorithm 2. For each member of the grouping attribute, the grouping algorithm performs transformation $\tilde{T_L}$ of the attribute member to a linguistic value, representing the crisp value of the grouping attribute. The value (tuple) is then assigned to a group, represented by a fuzzy set, based on the assigned label. In such a way, each of the attribute members is assigned to a group that fits the most closely. The time complexity of Algorithm 2 depends again on the number of tuples N within the dimension and the number of linguistic values v that should be checked. The creation of groups requires v steps, then we transform the value of a grouping attribute for each tuple (N) and transform it to a linguistic value (which also requires v steps). The theoretical complexity is then $O(v + Nv)$. Since, in most cases, v is low, the practical time complexity will be again $O(N)$.

For example, fuzzy grouping by the dimensional attribute *BMI* from the *Patient* dimension defined over the Hive table *pima_diabetes* with the *vBMI* linguistic variable defined, as presented in Fig. 2, for the part of the table presented in Table I produces the result presented in the table shown below. The example implements the following query formulated in natural language: *Show me the number of patients in particular (fuzzy) groups of the BMI index.*

BMI	Count
Underweight	1
Normal	3
Overweight	3
Obese	2.

The SQL-like form of the query would look as follows (note that this query operates on the entire *pima_diabetes* table, not on the dimension that can be identified within its structure):

```
SELECT bmiToLing(BMI) AS BMI,
  COUNT(*) AS Count
FROM pima_diabetes
GROUP BY bmiToLing(BMI)      .
```

The first column of the produced report contains the linguistic value of the *vBMI* variable. The second column provides the aggregated value of the total number of patients falling into the fuzzy group.

Fuzzy grouping in real HiveQL is shown in Q3 in Table III. It uses the *bmiToLing* function, which is a part of the Fuzzy-Hive extension introduced in Section IV. Fuzzy grouping on dimensional attributes can be simple (with one attribute or one fuzzy term, as in the example above) or multilevel. The multilevel grouping can include fuzzy terms or a mixture of crisp attributes and fuzzy terms. To aggregate data, users can also use a rich collection of Hive's built-in aggregate functions. The collection of aggregate functions that can be used in HiveQL includes *count*, *sum*, *avg*, *max*, *min*, *variance*, *stdev*, *covariance*, *percentile*, and many others.

In the cases when the experts' knowledge cannot be applied, e.g., due to lack of standards for the domain of the processed attributes, other algorithms can be implemented (e.g., fuzzy C-means) to support fuzzy grouping operations. However, this may require the usage of external frameworks, such as Apache Pig, that allow extraction and transformation of data.

E. Fuzzy Join

In the big data lake, some data required for fast analytics may be spread across many files, and it is challenging to create a single Hive table for them. Other analysis tasks that rely on a single Hive table may require data enrichment from other Hive tables. These cases lead to the necessity of joining data. Joining can be implemented in a standard (i.e., precise) or fuzzy way. Fuzzy joining allows combining not only corresponding data (based on the equality of values of a specified key), but also values that are similar to each other (i.e., by finding elastic relationships between values). Let us consider two Hive tables R of the schema $X_R = A, R_1, R_2, \ldots, R_m$ and S of the schema $X_S = A, S_1, S_2, \ldots, S_n$, where attribute A is a common attribute for both tables

$$\{A\} = X_R \cap X_S \tag{10}$$

$R_1, R_2, \ldots, R_m$ are remaining attributes of the Hive tables R, and $S_1, S_2, \ldots, S_n$ are remaining attributes of the Hive table S.

Fuzzy equijoin on the tables R and S can be implemented by representing values of joining attributes as fuzzy numbers (for the clarity of presentation, we provide the definition only for a pair of joining attributes)

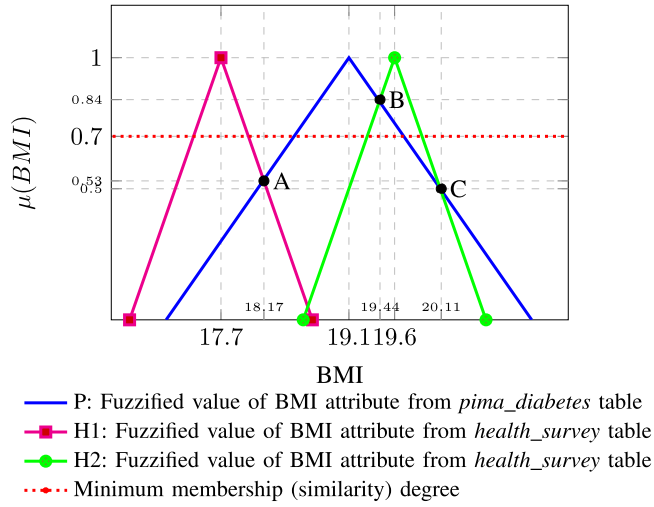

Fig. 3. Graphic representation of a fuzzy join performed through fuzzification of values of the BMI attributes from two tables: case (H1-P)—the tuples will not be joined (due to point A), case (H2-P)—the tuples will be joined (due to point B).

$$R \underset{R.A \approx S.A}{\tilde{\bowtie}^{\lambda}} S = \Pi_{X_R \cup X_S \cup X_\sim}(\tilde{\sigma}^{\lambda}_{\varphi_1(R.A) \approx \varphi_2(S.A)}(R \times S)) \tag{11}$$

where $\underset{R.A \approx S.A}{\tilde{\bowtie}^{\lambda}}$ is the symbol used for the fuzzy join operation by representing joining attributes $R.A, S.A$ as fuzzy numbers, $\Pi_{X_R \cup X_S \cup X_\sim}$ is the regular projection operation, $X_R \cup X_S \cup X_\sim$ is the new schema for the produced relation (table) containing attributes of both relations R and S, and additional attributes produced by the fuzzy join (represented by the schema $X_\sim$), $\tilde{\sigma}^{\lambda}_{\varphi_1(R.A) \approx \varphi_2(S.A)}$ is the fuzzy selection operation as defined earlier, λ is the minimum membership (similarity) degree, and $R \times S$ is the Cartesian product of the relation R with the relation S, $\{\varphi_1, \varphi_2\}$ are functions used for fuzzifying joining attributes A from both relations R and S. Each such function can be defined as follows:

$$\varphi(A) : t(A) \longrightarrow \{(t(A), \mu_\varphi(t(A)))\} \tag{12}$$

where $\mu_\varphi(t(A))$ is the membership function defining a fuzzy set for each value of the attribute A in tuple t of any relation.

The set $X_\sim$ of additional attributes produced by the fuzzy equijoin contains the similarity degree for fuzzified values of joining attributes, which is calculated, as presented in Fig. 3, for values of BMI attributes from both sample tables.

In this example, we perform fuzzy joining of the *pima_diabetes* (PD) and *health_survey* (HS) Hive tables based on the fuzzified values of the BMI attribute $\text{PD} \underset{\text{PD.BMI} \approx \text{HS.BMI}}{\tilde{\bowtie}^{0.7}} \text{HS}$. The example implements the following query formulated in natural language: *Show me the values of glucose (from PD) and cholesterol (from HS) for patients with similar BMI index*. The SQL-like form of the query would look like as follows (note that this query operates on the entire Hive tables, not just on the

dimensions identified within their structures):

```
SELECT PD.BMI, HS.BMI, MemDg, glucose,
  chol AS cholesterol
FROM pima_diabetes PD
  FJOIN health_survey HS
WHERE fequals(PD.BMI, 2, HS.BMI, 1) >= 0.7;
```

Values of the BMI from the first table are fuzzified using a spread equal to 2, and from the second table, a spread equal to 1. To accomplish the join, for each pair of tuples from both tables we calculate, an intersection of membership functions representing fuzzy sets defined for values of the BMI attributes (*fequals* function). Then, we check whether the membership degrees for the calculated intersection points meet the condition of the minimum membership degree. In Fig. 3, we assumed a minimum membership degree $\lambda = 0.7$. Partial results of the fuzzy join performed in such a way for both sample tables are presented in the table below– - the first two columns hold values of the BMI attribute from both tables, the third column holds the calculated membership degree, the remaining attributes show the most interesting data (for the sake of clarity, we do not show all attributes from both joined tables).

PD.bmi	HS.bmi	membershipDg	glucose	cholesterol
17.21	17.7	0.84	85	44
23.32	23.8	0.84	183	118
19.10	19.6	0.83	78	264
25.60	29.1	0.87	116	384
43.16	43.9	0.75	137	600

Fuzzy joining performed according to this method in real HiveQL query is shown in Q7 in Table III. It uses the *fequals* function, which is a part of the FuzzyHive extension introduced in Section IV.

F. Fuzzy Join With Linguistic Variables

Fuzzy join can also be implemented through fuzzy transformation $\tilde{T}_L$ of crisp values of the joining attributes A from both relations R and S with a predefined linguistic variable L. Labels then represent transformed values of joining attributes, and a regular join can be performed on the linguistic counterparts of the joining attributes. For the relations R and S, we can define the fuzzy equijoin with the use of linguistic variable L as follows:

$$R \underset{R.A \approx S.A}{\tilde{\bowtie}^{L}} S = \tilde{\Pi}_{X_R \cup X_S \cup X_\sim}(\sigma_{\tilde{T}_L(R.A)=\tilde{T}_L(S.A)}(R \times S)) \quad (13)$$

where $\underset{R.A \approx S.A}{\tilde{\bowtie}^{L}}$ is the symbol used for the fuzzy joint operation performed through fuzzy transformation with linguistic variable L, $\tilde{\Pi}_{X_R \cup X_S \cup X_\sim}$ is the extended projection with a fuzzy transformation defined earlier, $\sigma_{\tilde{T}_L(R.A)=\tilde{T}_L(S.A)}$ is the standard selection operation [56] with the filtering (selection) condition $\tilde{T}_L(R.A) = \tilde{T}_L(S.A)$, which defines the join condition for the fuzzy join operation.

For example, the table below shows partial results of the fuzzy join $\text{PD} \underset{\text{PD.BMI} \approx \text{HS.BMI}}{\tilde{\bowtie}^{v\text{BMI}}} \text{HS}$ performed with the *vBMI* linguistic variable defined in Fig. 2.

lingBMI	PD.bmi	HS.bmi	glucose	insulin	chol	pulse
Underweight	17.21	17.7	85	0	44	76
Normal	18.06	19.6	148	0	264	76
Normal	18.06	23.8	148	0	118	72
Normal	23.32	19.6	183	0	264	76
Normal	23.32	23.8	183	0	118	72
Normal	19.10	19.6	78	88	264	76
Normal	19.10	23.8	78	88	118	72
Overweight	28.11	29.1	89	94	384	60
Overweight	28.11	25.2	89	94	98	72
Overweight	25.60	29.1	116	0	384	60
Overweight	25.60	25.2	116	0	98	72
Overweight	27.81	29.1	115	543	384	60
Overweight	27.81	25.2	115	543	98	72
Obese	43.16	32.1	137	168	288	56
Obese	43.16	44.9	137	168	600	80
Obese	35.33	32.1	190	0	288	56
Obese	35.33	43.9	190	0	600	80

The SQL-like form of the query would look like as follows:

```
SELECT bmiToLing(PD.BMI) AS lingBMI, PD.BMI,
  HS.bmi, glucose, insulin, chol, pulse
FROM pima_diabetes PD
  JOIN health_survey HS
WHERE bmiToLing(PD.BMI) = bmiToLing(HS.BMI);
```

Values of the BMI attribute from both tables were first assigned to their linguistic counterparts (fuzzy sets) they fit the most. The *bmiToLing* function from the FuzzyHive extension introduced in Section IV was used for this purpose. An example of real fuzzy joining performed in HiveQL query according to this method is shown in Q6 in Table III.

Both types of fuzzy join operations can be performed on simple (with one attribute from each of the joined Hive tables, as in the examples above) or complex joining conditions. The complex joining conditions can include fuzzy joining conditions or a mixture of precise and fuzzy conditions.

IV. Implementation

FuzzyHive is a general-purpose library that allows for fuzzy querying in the Hive environment through extensions introduced to Hive query language (HiveQL). Additional definitions of domain-specific elements, such as definitions of linguistic variables and values, are provided by a user in separate files. This section gives quick information on, first, how to register the FuzzyHive library in Hive, second, which modules and functions are exposed for fuzzy querying in HiveQL, and third, what is the data organization in Hive.

A. Registering Library in Hive

Before the FuzzyHive library can be used, it should be registered in the Hive environment. A generalized installation and registration process is presented in Fig. 4.

FuzzyHive library is delivered in the form of library source code that must be compiled to the . *jar* file. This allows to add custom, specialized, and dedicated definitions of linguistic variables and their linguistic values. These definitions are stored in JSON files, such as the one presented in Appendix A. The compilation step can be skipped, if no custom definitions are added.

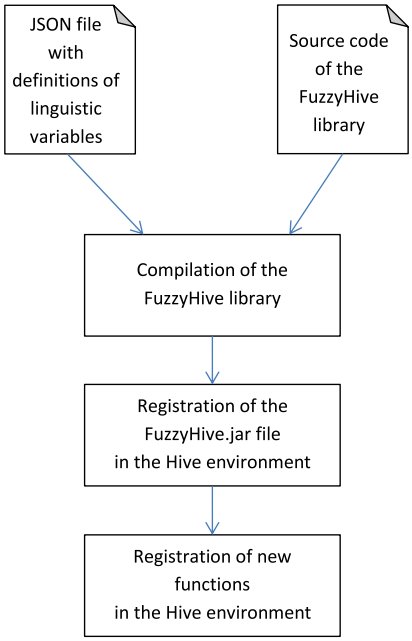

Fig. 4. Overview of registering and executing the FuzzyHive library in the Hadoop/Hive environment.

After compilation, the library should be registered in the Hive environment together with appropriate call specifications for all functions that will be invoked from the HiveQL queries. This step is shown in Appendix B.

B. FuzzyHive Modules

The FuzzyHive library contains various component modules exposing different classes, functions, and data types for fuzzy data processing. The most important are as follows.

1) *TrapezoidalMembershipFunction*—A class that allows creating trapezoidal and triangular membership functions to represent particular fuzzy sets or fuzzy values; it is advantageous when defining linguistic values for linguistic variables in JSON files.
2) *Around*—A function that assesses how much a given numeric value of the dimensional attribute matches to a fuzzy value represented by a trapezoidal membership function.
3) *membership*—A function that allows calculating membership degree of a given value of a dimensional attribute to a fuzzy set.
4) *fuzzyAnd*—A function that implements a fuzzy variant of the conjunction (AND) operator, used to check how much the complex filtering criteria on dimensional attributes are satisfied according to minimum t-norm.
5) *fuzzyOr*—A function that implements a fuzzy variant of the disjunction (OR) operator, used to check how much the complex filtering criteria on dimensional attributes are satisfied according to maximum t-conorm.
6) *fequals* (in particular, *FuzzyEquals*)—A function that determines the degree of equality of two fuzzy sets; it is beneficial while performing a fuzzy join on dimensional attributes.
7) Other functions that are dynamically created by code generator for the declared and defined linguistic values (e.g., *BMI_Low* and *BMI_Normal* for the sample JSON file presented in Appendix A).

C. Data Organization

Hive organizes data in databases. In Hive, a database is seen as a catalog or a namespace that separates data warehouse tables and other data units to avoid name conflicts [57]. Hive databases logically organize production tables into groups. Within databases, data are stored in tables that point to files located in the files system (HDFS). There can be various storage formats used for the files, such as given as follows.

1) Text files—Various types of text formats, including row delimited formats, e.g., comma-separated values and tab-separated values files.
2) Avro files—A compact, binary data format that provides a schema to the data to enable full processing of that data without code generation and facilitates the development of generic data-processing pipelines.
3) Parquet files—a compressed, columnar data representation format that supports nested data.
4) Others.

The three mentioned file formats were used in our experiments presented in Section V.

V. Experimental Results

The fuzzy extension to HiveQL was tested on the computer cluster created in the Amazon Web Services cloud platform. We used Amazon Elastic MapReduce (5.28.1 release) to set up the experimental environment containing Hadoop ver. 2.8.5, Hive ver. 2.3.6, Tez ver. 0.9.2. Particular components of the query execution environment had the following characteristics.

1) The computer cluster for data processing contained one name node hosted on the virtual machine of the m5.xlarge size 4 vCores, 16 GiB memory, EBS storage: 64 GiB on Intel Xeon Platinum 8000 series processor with Turbo CPU clock speed of up to 3.1 GHz with support for the Intel AVX-512 instruction set.
2) The data processing cluster contained seven data nodes hosted on the virtual machine of the m5.xlarge size.
3) Data tables had sizes ranging from one million to one billion rows.
4) Data were stored as standard text files (row text format), and also in Avro and Parquet formats.
5) Query execution was performed with the use of the Tez processing engine.

In our experiments, we particularly wanted to verify the following.

1) How the execution time of fuzzy analytical queries depends on the number of data?
2) How performing various types of fuzzy operations influences the execution time of queries?
3) How the selected storage format influences the execution time?

4) How the application of selected fuzzy operations affects the effectiveness of analytical HiveQL queries?

Our experiments were conducted with two published benchmark datasets—Pima Indians Diabetes Database [54] from UCI Machine Learning and the Third National Health and Nutrition Examination Survey dataset [55] collected by the United States Department of Health and Human Services, NHS. Partial data from both datasets were presented in Section III. To evaluate the effectiveness of the methods, we used the original data from both datasets. To assess time performance, we multiplied the data to several larger sizes by copying the records and modifying numerical values randomly.

A. HiveQL Analytical Queries

Experiments were performed with queries presented in Table III. Query Q1 consists of a simple fuzzy filtering condition on the dimensional attribute *BMI around 28* to find patients having a BMI around 28 with the minimum membership degree $\lambda = 0.7$. The fuzzy value *around 28* for the BMI attribute is defined with the use of the triangular membership function. Query Q2 performs fuzzy filtering but with the use of linguistic value. It finds underweight patients (concerning the BMI dimensional attribute). The linguistic value is represented by the `bmi_underweight` function, which was earlier created and registered in the Hive environment (as shown in Appendix B). Query Q3 performs fuzzy grouping according to a linguistic variable. It counts patients in particular groups of BMI defined by the `vBMI` linguistic variable (as in Appendix A). The `bmiToLing` function transforms crisp value of the `bmi` attribute to a fuzzy set according to Algorithm 1 and definition of the `vBMI` linguistic variable. Query Q4 contains complex filtering criteria on dimensional attributes composed of two simple conditions logically connected with `fuzzyor` operator. It displays patients whose BMI is around 28 or age is around 50. Query Q5 combines several operations of logical conjunction and logical disjunction. These operations are implemented in `fuzzyand` and `fuzzyor` functions. The query displays patients whose BMI is normal and glucose level is normal or who are obese or underweight. Query Q6 performs fuzzy join with the linguistic variable `vBMI`. It joins two datasets on similar BMI values (values assigned to the same BMI category or group). Query Q7 performs the fuzzy join through fuzzification of numeric values of BMI attributes from joined datasets and finding similar (matching) rows with a degree of similarity greater than 0.7. The query invokes the `fequals` function that fuzzifies the values of provided attributes and checks the similarity of fuzzy values.

B. Impact of the Increase in the Number of Data and Storage Format on the Query Execution Time

In Fig. 5, we can observe how the execution time of queries Q1–Q7 depends on the number of rows processed for various adopted data storage formats (Text, Avro, and Parquet). The experiments were conducted on tables with sizes ranging from one million (10^6) to one billion (10^9) rows. Queries were executed with the use of the Tez query execution engine. For queries Q1–Q5, we can observe that querying datasets of single millions of rows (e.g., one million) with fuzzy techniques is performed within several seconds. The execution times significantly increase from single seconds to minutes when querying datasets containing 100 million rows and one billion rows. Query Q4 has one fuzzy logical operation (implemented through the *fuzzyOr* function, whereas the query Q5 has three invocations of fuzzy logical operations: two calls to *fuzzyOr* and one to *fuzyAnd*, which doubles the execution time of the query Q5 in comparison to the execution time of the Q4 query. The execution times for Text format and Parquet format are similar for large datasets and are much shorter than for Avro format. The join operation performed in queries Q6 and Q7 causes much longer execution times—around 90 s for Q6 and 56 s for Q7 when querying a dataset with one million rows. Then, while increasing the amount of data, the execution times drastically increase to minutes or hours. Such execution times were expected and resulted from the need to process all pairs of values in the Cartesian product of the involved tables. Execution of queries Q6 and Q7 for larger amounts of data ended up with errors, which were probably caused by the number of join operations that were preceded by costly sort operations performed by the Hadoop processing engine.

C. Effectiveness of Fuzzy Operations

The effectiveness of fuzzy operations while querying on dimensional attributes was assessed with several measures. These measures reflect the enrichment to the result set provided by the applied fuzzy operations and express the enrichment quality.

One of the basic measures is the selectivity of a query. The selectivity reflects the percentage of tuples for which the HiveQL query (with fuzzy filtering conditions on dimensional attributes) is expected to return a true value. The selectivity shows the ratio between the cardinality of the output result set R_O after applying (fuzzy or crisp) filtering/join condition on a dimension and the cardinality of the processed input Hive table R

$$\text{Selectivity} = \frac{\text{card}(R_O)}{\text{card}(R)} \tag{14}$$

where card returns the number of rows for the given Hive table.

Relative cardinality, as defined in [58], is expressed as follows:

$$\text{RelCard}(R_O^F) = \frac{\sum_{t \in R_O^F} \mu_{R_O^F}(t)}{\sum_{t \in R} \mu_R(t)} \tag{15}$$

where R_O^F is the output result set of the fuzzy query, R is the input Hive table, and t is a processed tuple.

The flexibility of the fuzzy query Q_F reflects the contribution of additional rows R_O^Δ returned by the fuzzy query (concerning the rowset R_O^C returned by a crisp query) in the output result set R_O^F of the fuzzy query

$$\text{Flex}(Q_F) = \frac{\text{card}(R_O^\Delta)}{\text{card}(R_O^F)} = \frac{\text{card}(R_O^F \setminus R_O^C)}{\text{card}(R_O^F)}. \tag{16}$$

Precision expresses the relevance of the output result set R_O^F of a fuzzy query. Fuzzy precision is calculated according to the

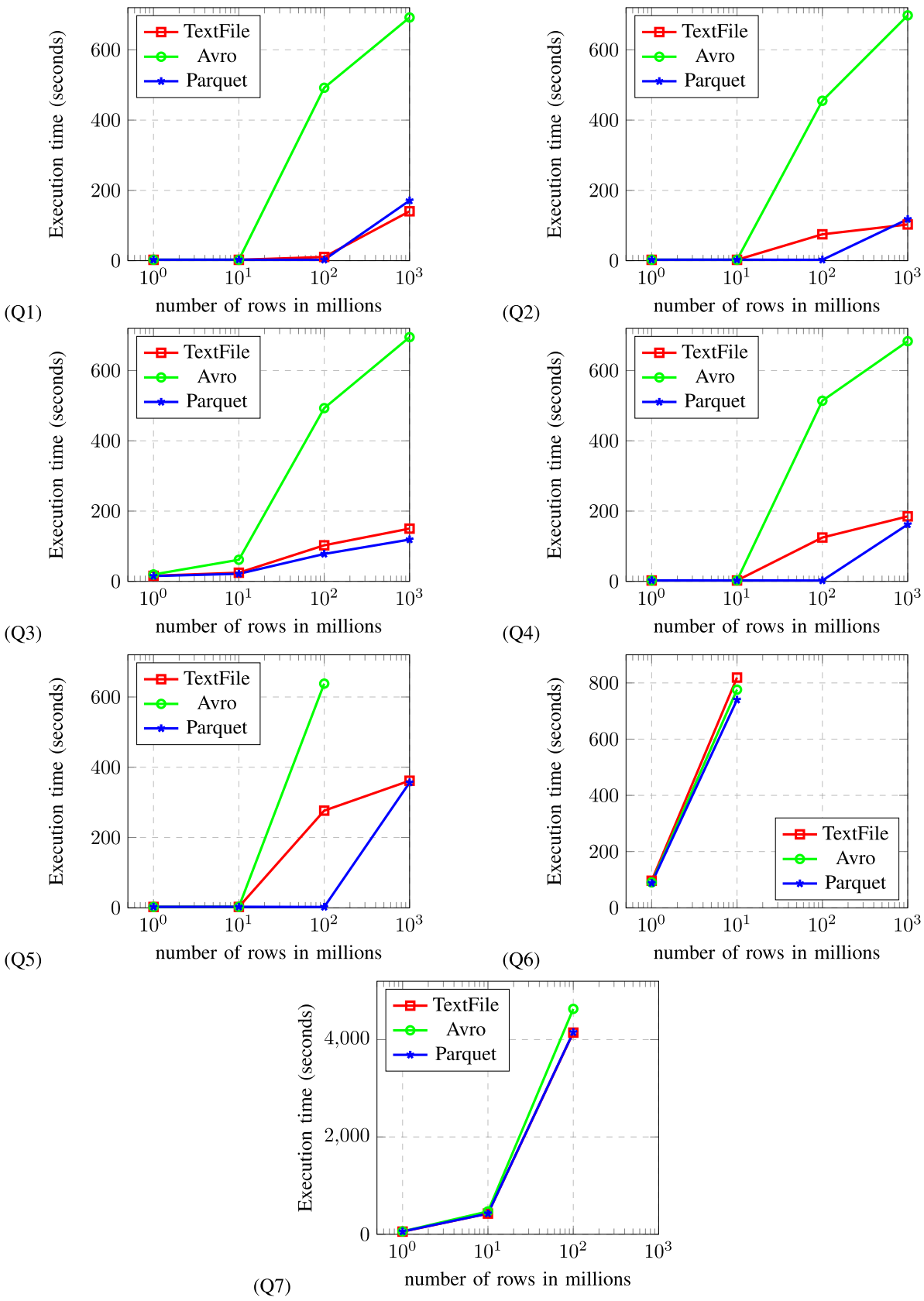

Fig. 5. Execution time for queries Q1–Q7 depending on the volume of data for selected data storage formats (TextFile, Avro, and Parquet).

TABLE IV
EFFECTIVENESS OF FUZZY QUERYING IN Q1

λ	Selc	RCard	Flex	FPrec
1.0	0.011	0.011	0.000	1.000
0.7	0.211	0.193	0.950	0.916

TABLE V
EFFECTIVENESS OF FUZZY QUERYING IN Q2

λ	Selc	RCard	Flex	FPrec
1.0	0.067	0.067	0.000	1.000
0.7	0.222	0.191	0.700	0.860

TABLE VI
EFFECTIVENESS OF FUZZY QUERYING IN Q4

λ	Selc	RCard	Flex	FPrec
1.0	0.133	0.133	0.000	1.000
0.7	0.322	0.293	0.545	0.910

following formula [59]:

$$\text{FPrec} = \frac{\sum_{t \in R} \mu_{R_O^F}(t)}{\text{card}(R_O^R)} \tag{17}$$

where R_O^R denotes the set of relevant (correct or expected) rows.

Results of effectiveness assessment for selected queries Q1, Q2, and Q4 that perform fuzzy filtering on dimensional attributes with $\lambda = 0.7$ are shown in Tables IV–VI and compared to their crisp counterparts (i.e., $\lambda = 1.0$). By making the queries more elastic, we increased the values of selectivity and flexibility measures. The flexibility for crisp queries is 0.0 and increases with the increasing value of the selectivity for the same λ, as well as in relation to the selectivity of a crisp query. If the selectivity of the crisp query is low, the increase is usually more significant. The relative cardinality, which for $\lambda = 1.0$ is the same as selectivity value, grows as well together with decreasing λ, but at a slower pace. This is because additional tuples that enrich the output result set contribute with various (lower than 1.0) membership degrees. Fuzzy precision decreases with the increased flexibility, which is expected, but stays at a relatively high level, proving that only tuples with high membership degrees are qualified for the produced result set.

Fig. 6 shows the CPU overhead for the execution of fuzzy queries Q1–Q5, for which we could successfully finish the execution process. Results show that the overhead was between 3% and 30%, which is acceptable considering that fuzzy queries require performing additional operations associated with transforming each of the processed tuples.

VI. DISCUSSION

Fuzzy extensions for Hive and HiveQL allow for scalable and flexible data warehousing that relies on declarative querying of large volumes of data stored in the Hadoop environment and its HDFS. This is important for many areas that utilize data warehouses and, at the same time, face challenges of big data, such as health and medical data analysis, fraud detection,

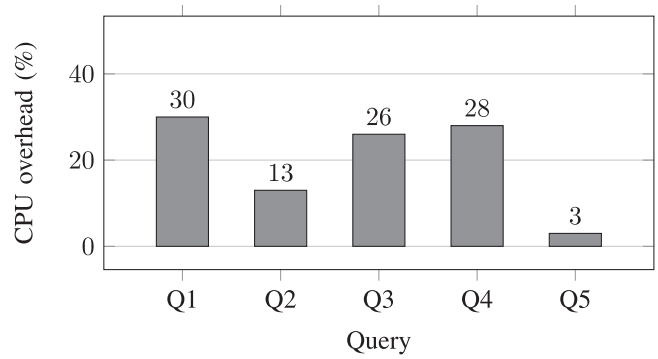

Fig. 6. Relative CPU overhead for execution of fuzzy queries.

sentiment analysis, risk modeling and management in financial institutions, and energy and power sectors. In those areas, providing the possibility of declarative querying can be a key to discover interesting facts over large portions of data almost immediately or in a short time. In this way, our solution responds not only to the value characteristic of big data but also to the velocity characteristic. Moreover, considering the possibility of Hive to define multiple schemas for the same data stored in the HDFS, it also satisfies the variety characteristic. Additional flexibility, achieved by the inclusion of fuzzy techniques in the HiveQL queries, allows a broader view of the analyzed data, which provides extra value. In such a way, the presented solution satisfies at least four out of five essential characteristics of the mentioned 5-V model of big data [52].

Since relational database systems, even those facilitating fuzzy querying, and traditional data warehouses do not allow processing and analysis of big data due to their rigid nature (data are structured), the use of frameworks that operate on top of Hadoop is necessary. Our solution allows this through the use of the Hive framework that generates MapReduce jobs as DAGs executed in Hadoop environment. In such a way, fuzzy queries can be performed in a highly distributed environment on partitioned data. In terms of the environment used, our solution is similar somehow to the MapReduce-based procedures for fuzzy data processing on Hadoop presented by Khorasani *et al.* [51]. However, Khorasani *et al.* showed a strictly procedural solution by implementing all fuzzy operations as MapReduce jobs, which would require adjusting implemented procedures to particular data ingestion cases and high programming skills to implement particular Map and Reduce phases (usually in the low-level Java API). The FuzzyHive library is independent of the execution engine. The fuzzy HiveQL queries can be executed on various execution engines supported by Hive, including MapReduce, Tez, and Spark. This can be achieved by changing the configuration of Hive, which is independent of the library (Fig. 1 in Section I).

Unlike the solution proposed by Khorasani *et al.* [51], our solution is also fully declarative, such as the relational database systems with fuzzy extensions to the SQL language, but scalable for big data at the same time. On the other hand, the procedural (imperative) approach, as proposed by Khorasani *et al.* [51], leaves the developer more control over the implementation and optimization of the provided functionality. The declarative

approach provides limited capabilities in this regard, which can be treated as a disadvantage of our solution. Unlike our previous solution for fuzzy querying in big data that relies on U-SQL language and Azure Data Lake service [52], [53], FuzzyHive is not bound to any commercial platform—Hive is managed by Apache Software Foundation and is an open-source framework. Our solution combines the portability of SQL-based applications (exactly HiveQL-based) with the flexibility of querying, including operations, such as fuzzy filtering, fuzzy grouping, and aggregating data, in created groups, projection with fuzzy transformation, and fuzzy join, to Hadoop and big data warehousing, which is a unique feature. The novelty lies mainly in immersing these operations in a new environment that has a grounded position in the big data ecosystem and scales well on the Cloud environments.

As we showed in our experiments, the fuzzy extensions for Hive scale quite well with the increasing volume of data for storing the data in simple text and Parquet files. Simple fuzzy filtering and grouping on dimensional attributes are executed faster than filtering operations using complex fuzzy selection criteria, which is expected due to additional calculations that have to be performed for compound filtering operations. Fuzzy join appeared to be a time-consuming operation comparing to other fuzzy operations performed. It leads to execution errors for large amounts of data, and we think it would be reasonable to perform this operation with prior data partitioning. For example, the data should be priorly filtered to narrow the joined datasets. Our results for the fuzzy join correspond to results obtained by Khorasani *et al.* [51], who also experienced the problem. The Avro storage format appeared to prolong the execution time comparing to the execution times of the same queries executed against tables stored in text and Parquet formats.

Finally, our experiments also showed that querying in HiveQL with the fuzzy extensions proposed in the FuzzyHive library increases the flexibility of queries. This allows for performing elastic data analyses in big data warehouses, where the schema is not predefined (such as in traditional data warehouses) but can be defined ad hoc according to the performed queries (*schema on read* approach). The flexibility introduced by fuzzy operations defined depend on the data, the distribution of values in particular attributes but always stays in relation to the crisp counterpart of the submitted query—the more rigid the query is (in terms of filtering conditions on dimensional attributes), the more flexible we can make it by the inclusion of fuzzy terms in the analysis process.

VII. Conclusion

Extending the Hive data warehousing framework by the possibility of fuzzy querying is another step in bringing flexibility to big data tools. Such extensions make big data warehousing more flexible and contribute to the portfolio of tools used by the community of people working with fuzzy sets and data analysis. The FuzzyHive library complements the spectrum of available solutions for fuzzy data processing and querying in large datasets. It addressed the challenges of big data by relying on the well-known and open-source Hadoop platform and HDFS. It is typically focused on batch querying and processing of data at rest, i.e., the inactive data collected mainly for reporting and analysis. However, in specific cases, it can also be used for querying data from data streams (data in motion) temporarily persisted in the HDFS data storage.

Future works will focus on developing similar capabilities that would support real-time (not batch) querying of data streams. This type of data has a completely different nature than data at rest that we process in Hive and requires flexible solutions that would operate in a dynamically changing environment.

Acknowledgment

The compiled version of the FuzzyHive library and its source code is available at GitHub.[1]

References

[1] P. Bosc and O. Pivert, *SQLf Query Functionality on Top of a Regular Relational Database Management System*. Heidelberg, Germany: Physica-Verlag HD, 2000, pp. 171–190.

[2] J. Kacprzyk and S. Zadrożny, *Data Mining Via Fuzzy Querying Over the Internet*. Heidelberg, Germany: Physica-Verlag HD, 2000, pp. 211–233.

[3] G. Bordogna and G. Psaila, "Customizable flexible querying in classical relational databases," in *Handbook of Research on Fuzzy Information Processing in Databases*, J. Galindo, Ed. Hershey, PA, USA: IGI Global, 2008, pp. 191–217.

[4] M. Hudec, "An approach to fuzzy database querying, analysis and realisation," *Comput. Sci. Inf. Syst.*, vol. 12, pp. 127–140, 2009. [Online]. Available: http://eudml.org/doc/253504

[5] B. Małysiak-Mrozek, S. Kozielski, and D. Mrozek, "Modern software tools for researching and teaching fuzzy logic incorporated into database systems," in *Proc. iNEER Int. Conf. Eng. Educ.*, Jul. 2010, pp. 1–8.

[6] B. Małysiak, D. Mrozek, and S. Kozielski, "Processing fuzzy SQL queries with flat, context-dependent and multidimensional membership functions," in *Proc. IASTED Int. Conf. Comput. Intell.*, Calgary, AB, Canada, 2005, pp. 36–41.

[7] B. Małysiak-Mrozek, D. Mrozek, and S. Kozielski, *Data Grouping Process in Extended SQL Language Containing Fuzzy Elements*. Advances in Intelligent and Soft Computing, vol 59, Berlin, Germany: Springer, 2009, pp. 247–256.

[8] L. Portinale and S. Montani, "A fuzzy logic approach to case matching and retrieval suitable to SQL implementation," in *Proc. 20th IEEE Int. Conf. Tools Artif. Intell.*, Washington, DC, USA, 2008, pp. 241–245.

[9] M. Delgado, C. Molina, D. Sanchez, A. Vila, and L. Rodriguez-Ariza, "A fuzzy multidimensional model for supporting imprecision in OLAP," in *Proc. IEEE Int. Conf. Fuzzy Syst.*, Jul. 2004, pp. 1331–1336.

[10] C. Molina, L. Rodriguez-Ariza, D. Sanchez, and M. A. Vila, "A new fuzzy multidimensional model," *IEEE Trans. Fuzzy Syst.*, vol. 14, no. 6, pp. 897–912, Dec. 2006.

[11] B. Małysiak-Mrozek, D. Mrozek, and S. Kozielski, *Processing of Crisp and Fuzzy Measures in the Fuzzy Data Warehouse for Global Natural Resources*. Lecture Notes in Computer Science, vol 6098, Berlin, Germany: Springer, 2010, pp. 616–625.

[12] R. Castillo-Ortega, N. Marín, and D. Sánchez, "Linguistic query answering on data cubes with time dimension," *Int. J. Intell. Syst.*, vol. 26, no. 10, pp. 1002–1021, 2011. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/int.20510

[13] G. A. Lara, M. Delgado, and N. Marín, *Fuzzy Multidimensional Modelling for Flexible Querying of Learning Object Repositories*. Berlin, Germany: Springer, 2013, pp. 112–123.

[14] D. Fasel, *Fuzzy Data Warehousing for Performance Measurement*. Cham, Switzerland: Springer, 2014.

[15] M. Martínez-Rojas, N. Marín, C. Molina, and M. Vila, "Cost analysis in construction projects using fuzzy OLAP cubes," in *Proc. IEEE Int. Conf. Fuzzy Syst.*, Aug 2015, pp. 1–8.

[16] A. Fayek, Ed., *Flexible Management of Essential Construction Tasks Using Fuzzy OLAP Cubes*. Bingley, U.K.: Emerald Publishing Limited, 2018, pp. 357–388.

[1] [Online]. Available: https://github.com/dmrozek/FuzzyHive.git

[17] M. S. Hidri, I. BenAli-Sougui, and A. Grissa-Touzi, "No-FSQL: A graph-based fuzzy NoSQL querying model," *Int. J. Fuzzy Syst. Appl.*, vol. 5, no. 2, pp. 54–63, 2016.

[18] A. Castelltort and A. Laurent, *Fuzzy Queries Over NoSQL Graph Databases: Perspectives for Extending the Cypher Language*. Cham, Switzerland: Springer, 2014, pp. 384–395.

[19] A. Castelltort and A. Laurent, "Extracting fuzzy summaries from NoSQL graph databases," in *Flexible Query Answering Systems (AISC)*, T. Andreasen, Ed. Berlin, Germany: Springer, 2016, pp. 189–200.

[20] A. Castelltort and A. Laurent, "Exploiting NoSQL graph databases and in memory architectures for extracting graph structural data summaries," *Int. J. Uncertainty, Fuzziness, Knowl.-Based Syst.*, vol. 25, no. 1, pp. 81–109, 2017.

[21] A. Castelltort and T. Martin, "Handling scalable approximate queries over NoSQL graph databases: Cypherf and the Fuzzy4S framework," *Fuzzy Sets Syst.*, vol. 348, pp. 21–49, 2018.

[22] A. B. Kacem and A. G. Touzi, "Towards fuzzy querying of NoSQL document-oriented databases," in *7th Int. Conf. Adv. Databases, Knowl., Data Appl.*, 2015, pp. 153–158.

[23] A. T. Azar and A. E. Hassanien, "Dimensionality reduction of medical big data using neural-fuzzy classifier," *Soft Comput.*, vol. 19, no. 4, pp. 1115–1127, 2015.

[24] Y. Li, R. Wang, and S. Shiu, "Interval extreme learning machine for big data based on uncertainty reduction," *J. Intell. Fuzzy Syst.*, vol. 28, no. 5, pp. 2391–2403, 2015.

[25] M. E. Bakry, S. Safwat, and O. Hegazy, "Big data classification using fuzzy K-nearest neighbor," *Int. J. Comput. Appl.*, vol. 132, no. 10, pp. 8–13, 2015.

[26] O. Hegazy, S. Safwat, and M. E. Bakry, "A MapReduce fuzzy techniques of big data classification," in *Proc. SAI Comput. Conf.*, Jul. 2016, pp. 118–128.

[27] P. Ducange, F. Marcelloni, and A. Segatori, "A MapReduce-based fuzzy associative classifier for big data," in *Proc. IEEE Int. Conf. Fuzzy Syst.*, Aug. 2015, pp. 1–8.

[28] S. del Río, V. López, J. M. Benítez, and F. Herrera, "A MapReduce approach to address big data classification problems based on the fusion of linguistic fuzzy rules," *Int. J. Comput. Intell. Syst.*, vol. 8, no. 3, pp. 422–437, 2015.

[29] V. López, S. del Río, J. M. Benítez, and F. Herrera, "Cost-sensitive linguistic fuzzy rule based classification systems under the MapReduce framework for imbalanced big data," *Fuzzy Sets Syst.*, vol. 258, pp. 5–38, 2015.

[30] M. Elkano, M. Galar, J. Sanz, and H. Bustince, "CHI-BD: A fuzzy rule-based classification system for big data classification problems," *Fuzzy Sets Syst.*, vol. 348, pp. 75–101, 2018. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S0165011417302774

[31] A. Segatori, F. Marcelloni, and W. Pedrycz, "On distributed fuzzy decision trees for big data," *IEEE Trans. Fuzzy Syst.*, vol. 26, no. 1, pp. 174–192, Feb. 2018.

[32] G. A. Papakostas, E. I. Papageorgiou, and V. G. Kaburlasos, "Linguistic fuzzy cognitive map (LFCM) for pattern recognition," in *Proc. IEEE Int. Conf. Fuzzy Syst.*, 2015, pp. 1–7.

[33] Y. Choi, H. Lee, and Z. Irani, "Big data-driven fuzzy cognitive map for prioritising IT service procurement in the public sector," *Ann. Oper. Res.*, vol. 270, pp. 75–104, 2018.

[34] J. Liu, Y. Chi, and C. Zhu, "A dynamic multiagent genetic algorithm for gene regulatory network reconstruction based on fuzzy cognitive maps," *IEEE Trans. Fuzzy Syst.*, vol. 24, no. 2, pp. 419–431, Apr. 2016.

[35] S. D'Onofrio, M. Wehrle, E. Portmann, and T. Myrach, "Striving for semantic convergence with fuzzy cognitive maps and graph databases," in *Proc. IEEE Int. Conf. Fuzzy Syst.*, 2017, pp. 1–6.

[36] S. A. Ludwig, "MapReduce-based fuzzy c-means clustering algorithm: Implementation and scalability," *Int. J. Mach. Learn. Cybern.*, vol. 6, no. 6, pp. 923–934, 2015.

[37] T. C. Havens, J. C. Bezdek, C. Leckie, L. O. Hall, and M. Palaniswami, "Fuzzy c-means algorithms for very large data," *IEEE Trans. Fuzzy Syst.*, vol. 20, no. 6, pp. 1130–1146, Dec. 2012.

[38] P. Su, C. Shang, and Q. Shen, "A hierarchical fuzzy cluster ensemble approach and its application to big data clustering," *J. Intell. Fuzzy Syst.*, vol. 28, no. 6, pp. 2409–2421, 2015.

[39] S. Prabha and P. Kola Sujatha, "Reduction of big data sets using fuzzy clustering," *Int. J. Adv. Res. Comput. Eng. Technol.*, vol. 3, no. 6, pp. 2235–2238, 2014.

[40] N. Bharill, A. Tiwari, and A. Malviya, "Fuzzy based clustering algorithms to handle big data with implementation on Apache Spark," in *Proc. IEEE 2nd Int. Conf. Big Data Comput. Serv. Appl.*, Mar. 2016, pp. 95–104.

[41] M. Prasad, Y. Lin, C. Lin, M. Er, and O. Prasad, "A new data-driven neural fuzzy system with collaborative fuzzy clustering mechanism," *Neurocomputing*, vol. 167, pp. 558–568, 2015.

[42] G. Peters and R. Weber, "DCC: A framework for dynamic granular clustering," *Granular Comput.*, vol. 1, no. 1, pp. 1–11, 2016.

[43] B. Zhang, S. Qin, W. Wang, D. Wang, and L. Xue, "Data stream clustering based on fuzzy C-mean algorithm and entropy theory," *Signal Process.*, vol. 126, no. C, pp. 111–116, Sep. 2016.

[44] V. Chitraa and A. Thanamani, "Web log data analysis by enhanced fuzzy c means clustering," *Int. J. Comput. Sci. Appl.*, vol. 4, no. 2, pp. 81–95, 2014.

[45] B. K. Tripathy, D. Mittal, and D. P. Hudedagaddi, *Hadoop With Intuitionistic Fuzzy C-Means for Clustering in Big Data*. Singapore: Springer, 2016, pp. 599–610.

[46] M. S. Hidri, M. A. Zoghlami, and R. B. Ayed, "Speeding up the large-scale consensus fuzzy clustering for handling big data," *Fuzzy Sets Syst.*, vol. 348, pp. 50–74, 2018. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S0165011417303901

[47] G. Smits, O. Pivert, R. R. Yager, and P. Nerzic, "A soft computing approach to big data summarization," *Fuzzy Sets Syst.*, vol. 348, pp. 4–20, 2018. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S0165011418300848

[48] I. G. Terrizzano, P. M. Schwarz, M. Roth, and J. E. Colino, "Data wrangling: The challenging yourney from the wild to the lake," in *Proc. 7th Biennial Conf. Innov. Data Syst. Res.*, Asilomar, CA, USA, Jan. 2015, pp. 4–7.

[49] R. Hai, S. Geisler, and C. Quix, "Constance: An intelligent data lake system," in *Proc. Int. Conf. Manage. Data*, New York, NY, USA, 2016, pp. 2097–2100. [Online]. Available: http://doi.acm.org/10.1145/2882903.2899389

[50] N. Miloslavskaya and A. Tolstoy, "Big data, fast data and data lake concepts," *Procedia Comput. Sci.*, vol. 88, pp. 300–305, 2016. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S1877050916316957

[51] E. S. Khorasani, M. Cremeens, and Z. Zhao, "Implementation of scalable fuzzy relational operations in MapReduce," *Soft Comput.*, vol. 22, no. 9, pp. 3061–3075, May 2018. [Online]. Available: https://doi.org/10.1007/s00500-017-2561-3

[52] B. Małysiak-Mrozek, M. Stabla, and D. Mrozek, "Soft and declarative fishing of information in big data lake," *IEEE Trans. Fuzzy Syst.*, vol. 26, no. 5, pp. 2732–2747, Oct. 2018.

[53] B. Małysiak-Mrozek, A. Lipińska, and D. Mrozek, "Fuzzy join for flexible combining big data lakes in cyber-physical systems," *IEEE Access*, vol. 6, pp. 69545–69558, 2018.

[54] R. S. Johannes, "Using the ADAP learning algorithm to forecast the onset of diabetes mellitus," *Johns Hopkins APL Tech. Dig.*, vol. 10, pp. 262–266, 1988.

[55] E. S. Ford, W. H. Giles, and W. H. Dietz, "Prevalence of the metabolic syndrome among US adults: Findings from the third National Health and Nutrition Examination Survey," *JAMA*, vol. 287, no. 3, pp. 356–359, 2002. [Online]. Available: https://doi.org/10.1001/jama.287.3.356

[56] E. F. Codd, "A relational model of data for large shared data banks," *Commun. ACM*, vol. 13, no. 6, pp. 377–387, Jun. 1970.

[57] E. Capriolo, D. Wampler, and J. Rutherglen, *Programming Hive*, 1st ed. Sebastopol, CA, USA: O'Reilly, 2012.

[58] L. A. Zadeh, "A computational approach to fuzzy quantifiers in natural languages," *Comput. Math. Appl.*, vol. 9, no. 1, pp. 149–184, 1983.

[59] B. Ziółko, "Fuzzy precision and recall measures for audio signals segmentation," *Fuzzy Sets Syst.*, vol. 279, no. C, pp. 101–111, Nov. 2015.

Bożena Małysiak-Mrozek (Member, IEEE) received the M.Sc. and Ph.D. degrees in computer science from the Silesian University of Technology, Gliwice, Poland, in 1995 and 2003, respectively.

She is currently an Assistant Professor with the Institute of Informatics, Silesian University of Technology, and a Member of the IBM Competence Center. Her scientific research interests include IoT, information systems, computational intelligence, bioinformatics, databases, big data, cloud computing, and soft computing methods.

Dr. Małysiak-Mrozek is an Organizing Committee Member of the International Conference Beyond Databases, Architectures, and Structures and the Co-Editor of 15 books devoted to databases and data processing. She is a Member of IEEE CI and SMC societies.

Jadwiga Wieszok received the M.Sc. degree in computer science from the Silesian University of Technology, Gliwice, Poland, in 2018.

She is currently with the Empirica Company, Wrocław, Poland, as a Software Developer and Scrum Master. Her research interests include algorithmic trading, big data, and agile software development.

Witold Pedrycz (Life Fellow, IEEE) received the M.Sc., Ph.D., and D.Sci. degrees in computer science from the Silesian University of Technology, Gliwice, Poland, in 1977, 1981, and 1984, respectively. He is currently a Professor and Canada Research Chair of computational intelligence with the Department of Electrical and Computer Engineering, University of Alberta, Edmonton, AB, Canada. He is also with the Systems Research Institute, Polish Academy of Sciences, Warsaw, Poland. In 2009, he was elected a foreign member of the Polish Academy of Sciences. He has authored or coauthored numerous papers in his research interests. He is an Author of 21 research monographs and edited volumes covering various aspects of computational intelligence, data mining, and software engineering. His main research interests include computational intelligence, fuzzy modeling, granular computing, knowledge discovery and data science, pattern recognition, data science, knowledge-based neural networks, and control engineering.

Dr. Pedrycz is vigorously involved in editorial activities. He is an Editor-in-Chief for the *Information Sciences* and WIREs Data Mining and Knowledge Discovery (Wiley), and a Co-Editor-in-Chief for the *International Journal of Granular Computing* (Springer) and *Journal of Data Information and Management* (Springer). He serves on an Advisory Board of the IEEE TRANSACTIONS ON FUZZY SYSTEMS. He is a Member of several editorial boards of international journals. In 2012, he was elected a Fellow of the Royal Society of Canada. He was the recipient of a prestigious Norbert Wiener Award from the IEEE Systems, Man, and Cybernetics Society in 2007. He was also the recipient of the IEEE Canada Computer Engineering Medal, a Cajastur Prize for Soft Computing from the European Centre for Soft Computing, a Killam Prize, a Fuzzy Pioneer Award from the IEEE Computational Intelligence Society, and the 2019 Meritorious Service Award from the IEEE Systems, Man, and Cybernetics Society. His current h-index is 114 (Google Scholar) and 87 on the list of top-h scientists for computer science and electronics.

Weiping Ding (Senior Member, IEEE) received the Ph.D. degree in computation application from the Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2013.

He was a Visiting Scholar with the University of Lethbridge, Lethbridge, AB, Canada, in 2011. From 2014 to 2015, he was a Postdoctoral Researcher with the Brain Research Center, National Chiao Tung University, Hsinchu, Taiwan. In 2016, he was a Visiting Scholar with the National University of Singapore, Singapore. From 2017 to 2018, he was a Visiting Professor with the University of Technology Sydney, Ultimo, NSW, Australia. He has authored or coauthored more than 80 research peer-reviewed journal and conference papers, including IEEE TFS, TNNLS, TCYB, TSMCS, TBME, TII, TEVC, TETCI, TCDS, and TITS, etc. His main research interests include data mining, granular computing, evolutionary computing, machine learning, and big data analytics.

Dr. Ding is currently the Chair of IEEE CIS Task Force on Granular Data Mining for Big Data. He is a Member of IEEE-CIS, ACM, CCAI, and Senior CCF. He is a Technical Committee Member on Soft Computing of IEEE SMCS, Granular Computing of IEEE SMCS, and Data Mining and Big Data Analytics of IEEE CIS. He currently serves on the Editorial Advisory Board of the *Knowledge-Based Systems* and Editorial Board of the *Information Fusion* and *Applied Soft Computing*. He serves as an Associate Editor for the IEEE TRANSACTIONS ON FUZZY SYSTEMS, *IEEE/CAA Journal of Automatica Sinica*, *Information Sciences*, *Neurocomputing*, *Swarm and Evolutionary Computation*, IEEE ACCESS, and *Journal of Intelligent & Fuzzy Systems*, a Co-Editor-in-Chief for the *Journal of Artificial Intelligence and System*. He is the Leading Guest Editor of Special Issues in several prestigious journals, including IEEE TRANSACTIONS ON EVOLUTIONARY COMPUTATION, IEEE TRANSACTIONS ON FUZZY SYSTEMS, *Information Fusion*, and *Information Sciences*. He has delivered more than 15 keynote speeches at international conferences. He has Co-Chaired several international conferences and workshops in the area of data mining, fuzzy decision-making, and knowledge engineering.

Dariusz Mrozek (Senior Member, IEEE) received the Ph.D. degree in computer science from the Silesian University of Technology (SUT), Gliwice, Poland, in 2006.

He is currently an Associate Professor and Head of the Department of Applied Informatics, SUT. He is the Author of 150+ papers published in conference proceedings and international journals, Author of two books on the use of big data analytics and high-performance computations in protein bioinformatics published by Springer, a Co-Editor of 15 other books devoted to databases and data processing, and an Editor of many special issues in reputable scientific journals. His research interests include IoT, parallel and cloud computing, databases, big data, and bioinformatics.

Dr. Mrozek is a Member of the IEEE Engineering in Medicine and Biology Society, the IEEE Systems, Man, and Cybernetics Society, and IEEE Cloud Computing Community. He has collaborated with qualified institutions by working on different research projects, such as Imperial College of London or Microsoft Research in the USA.